



UNIVERSITY OF TEHRAN

Signals & Systems second computer assignment

Image Processing

Author

Ermia Abbasi

Course Instructor

Prof. Rabiei

Date

July 10, 2026

Contents

1	Introduction	2
2	Problem 1: (Introduction to Fourier Transform in discrete 2-dimensional signals)	2
3	Problem 2: (Digital image processing and object selection using the properties of Fourier Transform)	8
4	Conclusion	15

Abstract

This project uses MATLAB to deal with some problems related to Image Processing. from image filters and masks to object detection.

1 Introduction

This is an introductory project about image processing for signals & systems course. we learn to generate histograms of images and interpret it. then we can take 2D Fourier transform and describe the output. we can even design some filters and masks. then we learn to apply these techniques on an image to change its background. the project has some small details which are Otsu's method ,Gaussian filters and so on.

2 Problem 1: (Introduction to Fourier Transform in discrete 2-dimensional signals)

I choose this Image of a cute cat to deal with in this part.



Figure 1. portrait of a cat

we want to find the size of this image. using these lines of code, we can do so.

```
1 img1 = imread('portrait-of-a-cat.jpg');  
2 size(img1)
```

we get three numbers as our output. 1067,1600,3. these mean our picture is 1600×1067 and it has all three color channels (Red, Green, Blue).

Now, we want to make it gray and change the format to double. Then we want to show them at the same page. we do so by these lines:

```
1 img2 = rgb2gray(img1);
2 img2 = im2double(img2);
3 tiledlayout(1,2);
4 nexttile
5 imshow(img1)
6 title('original image')
7 nexttile
8 imshow(img2)
9 title('grayscale image')
```

The output is as follows:

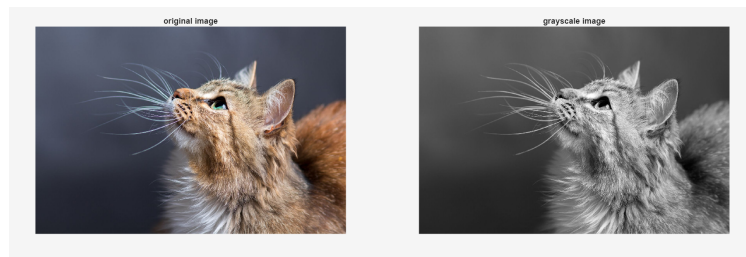


Figure 2. original and grayscale versions

The original image is a full-color RGB image but the processed image is a grayscale image. the first image is a 3D array but the second one is a 2D array, which means we have just one color channel. `img1` is unit8 but `img2` is double. `img1` pixel values are ranging from 0 to 255 but in `img2` pixel values are decimal numbers ranging from 0 to 1.

in order to generate the histogram we can use these lines of code:

```
1 figure;
2 imhist(img2)
3 title('histogram')
```

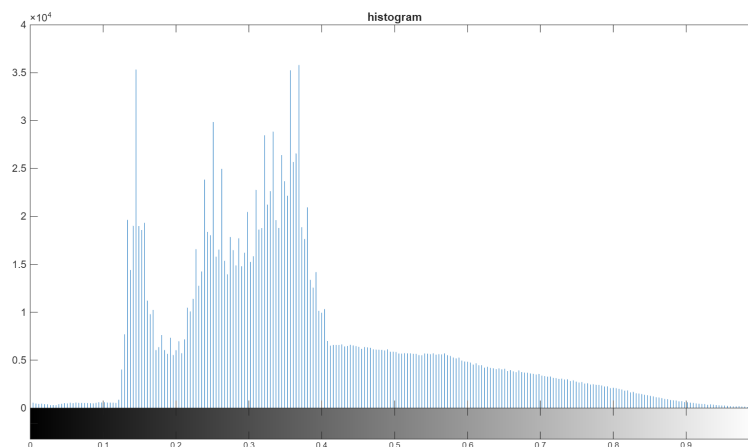


Figure 3. histogram of the grayscaled image

x axis is showing the color of pixels in grayscale mode and y axis is the number of pixels with that color.

contrast of the image is shown by the spread on the x axis of the histogram. therefore, low contrast images have narrower histograms and high contrast images have wider histograms. we can also have a mathematical indicator for this. we can use standard deviation by this code:

```
1 img_contrast = std2(img2);  
2 disp(['Contrast (Standard Deviation): ', num2str(img_contrast)]);
```

if this value is near zero we can say that the image is flat and it has almost no contrast. but if the value is higher we can say it has some contrast. for our image it was 0.17469.

now we want to take the 2D discrete Fourier transform and compare the image with values of Fourier transform. we use these lines of code:

```
1 F = fft2(img2);  
2 F_shift = fftshift(F);  
3 figure;  
4 tiledlayout(1,2);  
5 nexttile;  
6 imshow(img2);  
7 nexttile;  
8 imshow(log(1+abs(F_shift)),[]);
```

the output is like this:

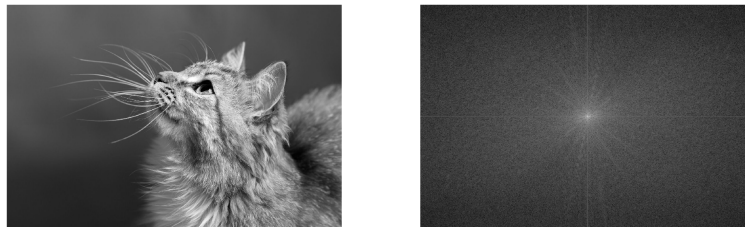


Figure 4. grayscaled and transformed images.

we are actually converting the spatial image into the frequency domain. the center of the frequency domain represents zero frequency (DC component) which is the average brightness of the image. near the center we have low frequencies and far from the center we have high frequencies. so, we took the Fourier transform and then we putted the zero frequency component in the center of the matrix. then we extracted the magnitude of complex coefficients and after that we showed them in a logarithmic scale because low freq components had large values and we couldn't show every part of them at the same time. the +1 is there to avoid $\log(0)$ in some points.

in order to create a mask we can use these codes:

```
1 [M, N] = size(img2);  
2 centerX = N / 2;  
3 centerY = M / 2;  
4 cutoff = 60;  
5 [X, Y] = meshgrid(1:N, 1:M);  
6 D = sqrt((X - centerX).^2 + (Y - centerY).^2);  
7 H_low = double(D <= cutoff);  
8 H_high = double(D > cutoff);  
9 figure; imshow(H_low);  
10 figure; imshow(H_high);
```

outputs are like this:

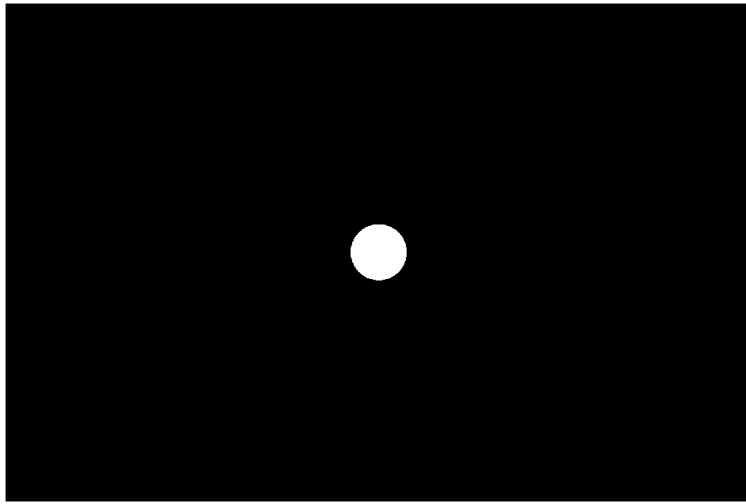


Figure 5. low-pass filter

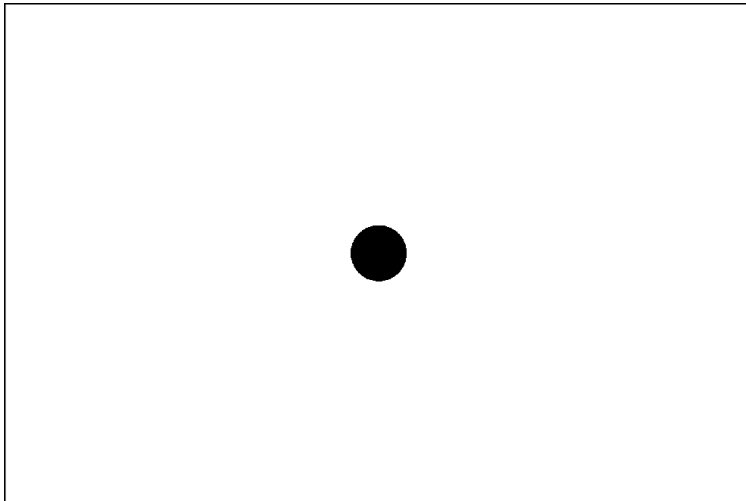


Figure 6. high-pass filter

The low-pass mask passes low frequencies around the center (which is zero frequency) and high-pass filter

passes all other frequencies except that range we selected. the meshgrid function generated a grid for us in order to have spatial coordinate on our canvas. the arguments are the ranges of the grid. we call these mask because they act like one. they hide somewhere of the picture and show other parts.

to apply the masks we write:

```
1 G_low_shift = F_shift .* H_low;  
2 G_high_shift = F_shift .* H_high;  
3 img_low = real(fft2(fftshift(G_low_shift)));  
4 img_high = real(fft2(fftshift(G_high_shift)));  
5 figure; imshow(img_low);  
6 figure; imshow(img_high);
```

the outputs are like this:



Figure 7. low-pass



Figure 8. high-pass

the reason we use `real()` in the code is that `ifft` and `fft` always produce some small imaginary values because of some round-off errors. in order to pass an argument to `imshow` it should be real. so, we discard the imaginary parts by `real()`.

the low-pass filtered image is blurred and smooth. we don't see the sharp details. the shapes and gradients are still there and we can see it's bright because it contains the DC value. on the other hand, the high-pass filtered image only has outlines and textures. the DC component is blocked too, therefore the image is dark.

Now, we do the same for this different picture:



Figure 9. high-pass filter applied.

using this code we did so:

```

1  img1 = imread( 'SS CA2\\Images\\image02.jpg' );
2  img2 = rgb2gray(img1);
3  img2 = im2double(img2);
4  F = fft2(img2);
5  F_shift = fftshift(F);
6  [M, N] = size(img2);
7  centerX = N / 2;
8  centerY = M / 2;
9  cutoff = 60;
10 [X, Y] = meshgrid (1:N, 1:M);
11 D = sqrt((X - centerX).^2 + (Y - centerY).^2);
12 H_high = double(D > cutoff);
13 G_high_shift = F_shift .* H_high;
14 img_high = real(ifft2(ifftshift(G_high_shift)));
15 figure;
16 tiledlayout(1,2);
17 nexttile;
18 imshow(img1);
19 nexttile;
20 imshow(img_high);

```

in a digital image. frequency represents the rate of change of pixels intensities with respect to distance. therefore, low frequencies are the parts which are smooth and has little intensity changes, but the high frequency parts are those parts in which we have sharp changes which are the boundaries of objects. our high-pass filter only lets the high-freq values to pass, therefore we just see the outlines.

3 Problem 2: (Digital image processing and object selection using the properties of Fourier Transform)

For this part I'm gonna use this image of a flying owl. It has all the properties we want.



Figure 10. a flying owl

the grayscale version is like this:



Figure 11. grayscale version of the owl

we do the fourier transform and high-pass filter: (with cutoff=115)



Figure 12. high-pass filter

till now, our codes were like this:

```

1  img1 = imread('bdic-report-template-latex-main\images\phil-robson-R0T-2U-O5fc-unsplash.jpg');
2  size(img1)
3  img2 = rgb2gray(img1);
4  img2 = im2double(img2);
5  F = fft2(img2);
6  F_shift = fftshift(F);
7  [M, N] = size(img2);
8  centerX = N / 2;
9  centerY = M / 2;
10 cutoff = 115;
11 [X, Y] = meshgrid(1:N, 1:M);
12 D = sqrt((X - centerX).^2 + (Y - centerY).^2);
13 H_high = double(D > cutoff);
14 G_high_shift = F_shift .* H_high;
15 img_high = real(fft2(fftshift(G_high_shift)));
16 figure;
17 imshow(img_high)

```

It's good to apply some gaussian filters at different stages of this process. i do it now:

```

1  img_blured = imgaussfilt(img_high, 0.3);

```

according to images histogram, we can apply a method to make the contrast much better for object detection. I do Otsu's method. the code is as follows:

```

1  th = otsuthresh(imhist(img_blured));
2  binary_img = (img_blured >= th);

```

but how does it work? it first calculates the image histogram to get the probability of each gray level occurring in the image. then for every potential threshold it splits the pixels into two groups, pixels with less than the threshold intensity and vice versa. it calculates the two classes percentages(w_1 & w_2) and averages(μ_1 & μ_2)

and average of entire image. then it finds a between-class variance:

$$\sigma_B^2(t) = w_1(t).w_2(t).(\mu_1(t) - \mu_2(t))^2$$

it calculates this for every possible threshold and selects the threshold that maximizes this.

now we use `bwareaopen` with value 40. the `bwareaopen` gets a binary image and a number. any object in the image that has less than that number of pixels connected together will be removed(blacked). (it can have a third argument that shows the connectivity of pixels.) the output is like this:

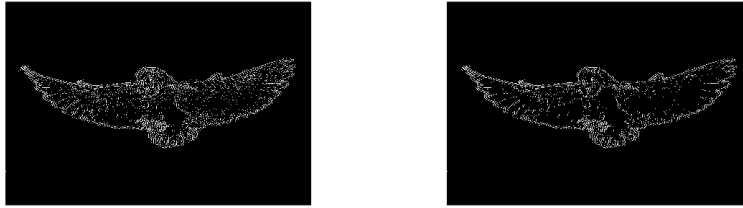


Figure 13. left one is the binarized image without noise-reduction. the right one is after noise-reduction.

the total code for this part is like this:

```
1 figure;
2 tiledlayout(1,2);
3 nexttile;
4 th = otsuthresh(imhist(img_blured));
5 binary_img = (img_blured >= th);
6 imshow(binary_img);
7 img4 = bwareaopen(binary_img,40);
8 nexttile;
9 imshow(img4);
```

in order to create a mask out of this image, we can do two for loops, one horizontally and one vertically and then with an ANDing we get the result we want.(this is because the object doesnt have any holes in it.)

```
1 [rows,cols] = size(img4);
2 img_hor = img4;
3 for i = 1:rows
4     first_white = 0;
5     last_white = 0;
6     for j = 1:cols
7         if img4(i, j) == 1
8             if first_white == 0, first_white = j; end
9             last_white = j;
10        end
11    end
12    if first_white > 0 && last_white > first_white
13        img_hor(i, first_white:last_white) = 1;
14    end
15 end
16
17 img_ver = img4;
18 for j = 1:cols
19     first_white = 0;
```

```
20 last_white = 0;
21 for i = 1:rows
22     if img4(i, j) == 1
23         if first_white == 0, first_white = i; end
24         last_white = i;
25     end
26 end
27 if first_white > 0 && last_white > first_white
28     img_ver(first_white:last_white, j) = 1;
29 end
30 end
31 img_com = img_hor&img_ver;
32 img_com = double(img_com);
```

right now, we should do sth to make the output smoother. i do another gaussian blur and another otsuthresh method:

```
1 img_com_blurred = imgaussfilt(img_com, 15);
2 th2 = otsuthresh(imhist(img_com_blurred));
3 blurred_binary_img = (img_com_blurred >= th2);
4 figure;
5 imshow(blurred_binary_img);
```

the output mask for this part is like this:



Figure 14. our smoothed mask

putting this on our owl image leads to this output:



Figure 15. owl with removed background

which is done by this loop:(for all three RGB channels.)

```
1 output = img1;
2 for c=1:3
3     channel = output(:,:,c);
4     channel(~blurred_binary_img) = 0;
5     output(:,:,c)=channel;
6 end
7 figure;
8 imshow(output);
```

to scale this with the background sky image and put it there we can do this:

```
1 background = imread('SS CA2\images/image03.jpg');
2 background = imresize(background,[size(output,1), size(output,2)]);
3 final = background;
4 for r = 1:rows
5     for c = 1:cols
6         if blurred_binary_img(r, c) == 1
7             final(r, c, :) = output(r, c, :);
8         end
9     end
10 end
11 figure;
12 imshow(final);
```

which gives this final result:



Figure 16. owl flying in a sky!

we can write everything as a matlab function and test it (the values can be changed for better results with different pictures.)

```

1  function final_img = replaceBackground(foreground_img, background_img)
2  img_gray = rgb2gray(foreground_img);
3  img_double = im2double(img_gray);
4
5  F = fft2(img_double);
6  F_shift = fftshift(F);
7  [M, N] = size(img_double);
8
9  centerX = N / 2;
10 centerY = M / 2;
11 cutoff = 115;
12 [X, Y] = meshgrid(1:N, 1:M);
13 D = sqrt((X - centerX).^2 + (Y - centerY).^2);
14 H_high = double(D > cutoff);
15
16 G_high_shift = F_shift .* H_high;
17 img_high = real(ifft2(ifftshift(G_high_shift)));
18
19 img_blurred = imgaussfilt(img_high, 0.3);
20 th = otsuthresh(imhist(img_blurred));
21 binary_img = (img_blurred >= th);
22
23 img4 = bwareaopen(binary_img, 40);
24 [rows, cols] = size(img4);
25
26 img_hor = img4;
27 for i = 1:rows
28     first_white = 0;
29     last_white = 0;
30     for j = 1:cols
31         if img4(i, j) == 1
32             if first_white == 0, first_white = j; end
33             last_white = j;
34         end

```

```
35 end
36 if first_white > 0 && last_white > first_white
37 img_hor(i, first_white:last_white) = 1;
38 end
39 end
40
41 img_ver = img4;
42 for j = 1:cols
43 first_white = 0;
44 last_white = 0;
45 for i = 1:rows
46 if img4(i, j) == 1
47 if first_white == 0, first_white = i; end
48 last_white = i;
49 end
50 end
51 if first_white > 0 && last_white > first_white
52 img_ver(first_white:last_white, j) = 1;
53 end
54 end
55
56 img_com = img_hor & img_ver;
57 img_com = double(img_com);
58 img_com_blurred = imgaussfilt(img_com, 15);
59 th2 = otsuthresh(imhist(img_com_blurred));
60 blurred_binary_img = (img_com_blurred >= th2);
61
62 output = foreground_img;
63 for c = 1:3
64 channel = output(:, :, c);
65 channel(~blurred_binary_img) = 0;
66 output(:, :, c) = channel;
67 end
68
69 resized_bg = imresize(background_img, [rows, cols]);
70
71 final_img = resized_bg;
72 for r = 1:rows
73 for c = 1:cols
74 if blurred_binary_img(r, c) == 1
75 final_img(r, c, :) = output(r, c, :);
76 end
77 end
78 end
79 end
80
81 img1=imread("bdic-report-template-latex-main\images\phil-robson-R0T-2U-O5fc-unsplash.jpg");
82 bg_img = imread("SS CA2\Images\image03.jpg");
83
84 result = replaceBackground(img1,bg_img);
85
86 figure;
87 imshow(result);
```

4 Conclusion

In order to select an object from an arbitrary image, we can use its grayscale version with a high pass filter. then we can do Otsu's threshold method and change some small details to generate a better output. after that, we are left with a mask that can be putted on our image and then the new image with removed background can be putted everywhere we want by just a simple for loop.